

[DataX] AI 辅助编写 workflow 脚本

练习题

习题一 编写配置文件

现在 Mysql 中有一张表，下面是表在 Mysql 中的字段名和类型关系

id : bigint

user_name: varchar(255)

create_time: datetime

请尝试用 GPT 编写一份将 mysql 表同步到 hdfs 的配置文件

习题二 编写参数化配置

依赖习题一

你可以想一下之前的配置文件中有什么地方是需要参数化的，现在你可以尝试在同一个 GPT 会话中，将之前的配置文件修改为一个参数化的配置文件。

参考答案

习题一 编写配置文件

第一轮提示

提示

现在 Mysql 中有一张表，下面是表在 Mysql 中的字段名和类型关系

id : bigint

user_name: varchar(255)

create_time: datetime

我想用 Datanux 将它同步到 hdfs 上，请帮我写一份配置文件。

答

Datanux 是一个数据同步工具，可以在多种数据源之间进行数据传输。下面是一个使用 Datanux 将 MySQL 数据同步到 HDFS 的配置文件示例：

```
{
  "job": {
    "setting": {
      "speed": {
        "channel": 4
      },
      "errorLimit": {
        "record": 0,
        "percentage": 0.02
      }
    },
    "content": [
      {
        "reader": {
          "name": "mysqlreader",
          "parameter": {
            "username": "your_username",
            "password": "your_password",
            "connection": [
              {
                "jdbcUrl":
"jdbc:mysql://your_mysql_host:your_mysql_port/your_database",
                "table": [
                  "your_table"
                ]
              }
            ]
          }
        }
      }
    ]
  }
}
```

```

        ]
    }
],
"column": [
    "id",
    "user_name",
    "create_time"
],
"splitPk": "id"
}
},
"writer": {
    "name": "hdfswriter",
    "parameter": {
        "defaultFS": "hdfs://your_hdfs_host:your_hdfs_port",
        "fileType": "text",
        "path": "/your_destination_directory_on_hdfs",
        "fileName": "your_destination_file_name",
        "column": [
            {
                "name": "id",
                "type": "bigint"
            },
            {
                "name": "user_name",
                "type": "string"
            },
            {
                "name": "create_time",
                "type": "timestamp"
            }
        ],
        "writeMode": "append",
        "fieldDelimiter": ",",
    }
}
}

```

```
}  
]  
}  
}
```

请将上述配置文件中的占位符（例如：your_username、your_password 等）替换为您实际的配置信息。

注意：如果您的 HDFS 集群启用了安全模式（Kerberos），则需要在“hdfswriter”参数中添加“kerberosKeytabFilePath”和“kerberosPrincipal”参数，并提供对应的 Keytab 文件路径和 Kerberos Principal。

反馈

GPT 给出了正确的配置，如果你不太理解其中的一些配置，还可以要求 GPT 进行详细的解释。

第 2 轮提示

提示

请详细解释一下刚才配置中的 job 部分

答

setting 部分是 Datanux 配置文件中的任务设置部分，主要用于控制任务执行的速度和允许的误差限制。以下是 **setting** 部分的详细解释：

1. **speed**: 速度相关设置。

- **channel**: 指定任务的并发度，即同时启动多少个线程进行数据同步。例如，这里设置为 4，表示有 4 个线程同时进行数据读取和写入。根据实际需求和硬件资源，可以适当调整此值以优化任务性能。

2. **errorLimit**: 错误限制相关设置。

- **record**: 允许的最大错误记录数。如果在数据同步过程中，错误记录数超过此值，任务将终止。例如，这里设置为 0，表示不允许有任何错误记录。可以根据实际情况调整此值。
- **percentage**: 允许的最大错误记录百分比。如果在数据同步过程中，错误记录百分比超过此值，任务将终止。例如，这里设置为 0.02，表示错误记录数不能超过总记录数的 2%。这个值可以根据实际需求调整。

setting 部分的配置可以根据实际需求和资源进行调整，以满足不同场景的数据同步要求。例如，可以提高 **channel** 值以提高任务并发度，从而提高数据同步速度；也可以适当调整 **errorLimit** 的值，以容忍一定数量的错误记录，确保任务的正常完成。

反馈

这里出现一个奇怪的问题，**record** 设置为 0，但 **percentage** 设置为 0.02，看上去是相互冲突的，你可以对这个问题继续追问。

第 3 轮提示

提示

record 设置为 0，但 **percentage** 设置为 0.02，是不是没什么意义，到底哪个会生效。

答

您提出了一个很好的问题。在这种情况下，**record** 和 **percentage** 两个错误限制条件会同时生效。任务将在满足任一条件时终止。

例如，配置文件中 **record** 设置为 0，表示不允许有任何错误记录。而 **percentage** 设置为 0.02，表示错误记录数不能超过总记录数的 2%。当数据同步过程中出现错误记录时，将检查这两个条件。

1. 如果出现了一个错误记录，那么 **record** 条件将触发，任务会立即终止。
2. 如果错误记录数没有超过总记录数的 2%，那么 **percentage** 条件暂时不会触发。

在这种配置下，由于 **record** 设置为 0，所以只要出现一个错误记录，任务就会立即终止，**percentage** 的设置实际上并不起作用。这种情况下，您可以根据实际需求调整 **record** 和 **percentage** 的值，以便在满足任务需求的同时容忍一定数量的错误记录。例如，可以将 **record** 设置为一个非零值，如 10，表示允许最多有 10 个错误记录。

反馈

可以看到讲解非常到位。

习题二 编写参数化配置

第 1 轮提示

提示

之前的配置文件每次运行会同步表中的所有数据，我现在希望每次运行的时候可以使用命令行参数指定一个日期，然后根据 `create_time` 字段，只同步指定日期创建的用户记录。

答

Datax 本身不支持直接通过命令行参数传递查询条件，但您可以使用 Shell 脚本实现此需求。以下是一个简化的示例：

1. 首先，在 Datax 配置文件中添加一个占位符（如：**{date}**）作为日期过滤条件：

```
{  
  ...  
  "reader": {  
    "name": "mysqlreader",  
    "parameter": {
```

```
...
    "querySql": [
        "SELECT id, user_name, create_time FROM your_table WHERE DATE(create_time)
= ' {date}'"
    ],
    ...
}
},
...
}
```

2. 创建一个 Shell 脚本（如：**run_datax.sh**），用于接收命令行参数并替换配置文件中的占位符：

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Usage: $0 <date>"
    exit 1
fi

DATE=$1
CONFIG_FILE="datax_config.json"
TEMP_CONFIG_FILE="temp_datax_config.json"

sed "s/{date}/${DATE}/g" $CONFIG_FILE > $TEMP_CONFIG_FILE

datax.py $TEMP_CONFIG_FILE

rm $TEMP_CONFIG_FILE
```

3. 赋予脚本可执行权限：

```
chmod +x run_datax.sh
```

4. 使用指定日期作为参数运行脚本：

```
./run_datax.sh "2023-05-09"
```

这样，每次运行脚本时，将根据提供的日期参数过滤 `create_time` 字段，并仅同步指定日期创建的用户记录。

请注意，这个示例仅适用于符合日期格式（如 “2023-05-09”）的参数。您可以根据需要自定义脚本以处理不同格式的日期参数。

反馈

可以看到 GPT 很快意识到这个功能需要用 SQL 的方式来完成，并给出了原先配置文件的修改方案。但是有一个问题是，GPT 说 DataX 不支持传递参数，这个就是错误的了。GPT 也给出了另外一种方案，就是先创建一个带参数的配置文件，然后通过一个调度脚本在运行时接收命令行参数，创建一个临时文件来做插值替换，最后交给 DataX 执行，在操作完成后，脚本还会将临时文件删除，这也算是一种可行的解决方案。